Socratic-DB

Tutor socratico per la progettazione di basi di dati relazionali

Gionata Massi

Indice

1 Introduzione . . . 5
1.1 Piani di verifica della sperimentazione . . . 5
1.2 Struttura del documento . . . 6

2 Stato dell'arte e inquadramento teorico . . . 7
2.1 Introduzione e contesto . . . 7
2.2 Sistemi di Tutorato Intelligente e didattica dei database . . . 7
2.2.1 Errori e misconcezioni nella formulazione di query SQL . . . 8
2.2.2 Modelli mentali e spiegazioni dei sistemi di dati . . . 8
2.2.3 Generazione di esercizi SQL con LLM e validazione . . . 8
2.3 LLM nell'istruzione informatica e tutoraggio dialogico . . . 8
2.3.1 Opportunità e rischi dell'uso generativo . . . 8
2.3.2 Tutoraggio dialogico, scaffolding e limiti degli LLM . . . 9
2.4 Architettura ibrida: controlli formali, esecuzione e dialogo . . . 9
2.5 Contributo della proposta e questioni aperte . . . 10
2.6 Quadro normativo e protezione dei dati . . . 10
2.7 Sintesi del posizionamento . . . 11

3 Specifica pedagogica . . . 13
3.1 Destinatari . . . 13
3.2 Prerequisiti . . . 13
3.3 Obiettivi di apprendimento . . . 13
3.4 Ruoli . . . 14
3.4.1 Docente . . . 14
3.4.2 LLM (tutor) . . . 14
3.4.3 Sistema deterministico (orchestratore e controlli) . . . 14
3.5 Verifica dell'apprendimento: prodotto, processo, trasferimento . . . 15
3.6 Rischi didattici e contromisure . . . 15

4 Protocollo didattico . . . 17
4.1 Il percorso: nove fasi in cinque consegne . . . 17
4.1.1 Sulla generazione automatica . . . 19
4.2 Errori oggettivi e decisioni progettuali . . . 19
4.3 Livelli di aiuto progressivi . . . 20
4.4 Ciclo di revisione . . . 20
4.5 Padronanza per competenza . . . 20
4.6 Precisazioni tecniche incorporate . . . 20
4.7 Metriche separate . . . 21
4.8 Sostenibilità per il docente . . . 21
4.8.1 Pacchetto minimo e pacchetto esteso . . . 21

4.8.2 Strumento d'autore (AI dietro le quinte) . . . 21
4.8.3 Catalogo di schemi-tipo . . . 21
4.8.4 Tracce d'esame come punto di partenza . . . 21
4.8.5 Report compatto . . . 22
4.8.6 Pre-screening delle motivazioni . . . 22
4.8.7 Ipotesi di carico (da misurare nel pilota) . . . 22

5 Architettura del sistema e prototipo tecnico . . . 23
5.1 Principi architetturali . . . 23
5.2 Architettura del prototipo . . . 23
5.2.1 Flusso operativo . . . 23
5.3 Hardware e scelta del modello . . . 23
5.4 Esecutore SQLite isolato . . . 24
5.5 Test automatici del sistema . . . 24
5.5.1 Test dei controlli deterministici . . . 24
5.5.2 Test sugli esercizi (Golden set di ricerca e schema canonico) . . . 24
5.5.3 Test del tutor LLM (Golden set) . . . 25
5.5.4 Verifica deterministica dei dati di test generati dalla LLM (fase 7, se abilitata) . . . 25
5.6 System prompt del tutor (Versione per il prototipo) . . . 25
5.7 GitLab / Forgejo come piattaforma di consegna e revisione . . . 26
5.8 Strumento d'autore per il docente . . . 26
5.9 Ipotesi di sistema avanzato (Post-sperimentazione) . . . 27

6 Specifiche dei formati e contratti dati . . . 29
6.1 Specifiche YAML per i modelli . . . 29
6.1.1 Modello Concettuale (`tipo: concettuale`) . . . 29
6.1.2 Modello Relazionale (`tipo: relazionale`) . . . 30
6.2 Struttura dei file di consegna dello studente . . . 31
6.3 Formato delle diagnosi strutturate . . . 32
6.4 Formato del report compatto di processo . . . 32

7 Quadro normativo, sicurezza e privacy . . . 35
7.1 Trattamento dei dati personali (GDPR) . . . 35
7.1.1 Adempimenti documentali . . . 36
7.2 Misure di sicurezza tecniche e organizzative . . . 36
7.3 Valutazione e quadro regolatorio sull'Intelligenza Artificiale (AI Act) . . . 36
7.4 Limiti del sistema e minacce alla validità . . . 37

8 Protocollo di sperimentazione empirica . . . 39
8.1 Fasi e disegno della ricerca . . . 39
8.2 Raccolta dati e metriche di processo . . . 40
8.3 Analisi statistica e criteri di successo . . . 40
8.3.1 Criteri di successo definiti a priori . . . 40
8.4 Etica, governance e consenso . . . 41

Bibliografia . . . 43

1. Introduzione

Sommario. Questo documento descrive una proposta di strumento e metodologia didattica basata sul metodo socratico, per aiutare gli studenti ad acquisire la capacità di progettare e implementare basi di dati relazionali, dall'analisi concettuale al modello logico. Il processo mira a condurre lo studente a produrre una base di dati completa, pertinente, minimale e intelligibile a partire da una descrizione testuale del problema. La costruzione della conoscenza procede come dialogo tra l'arte della domanda, affidata a modelli linguistici, e il rigore della struttura, verificato mediante controlli logico-simbolici. Le abilità modellistiche e di programmazione sono oggetto di esercizio e feedback durante il percorso; la sperimentazione valuterà l'efficacia della proposta sulla base delle interazioni con il sistema, raccolte e trattate in forma pseudonimizzata secondo le regole definite per lo studio.

Socratic-DB è un ambiente di apprendimento assistito dall'Intelligenza Artificiale Generativa per la scuola secondaria di secondo grado (ITE indirizzo *Sistemi Informativi Aziendali* e ITT indirizzo *Informatica*) e per corsi universitari introduttivi di basi di dati.

Il progetto nasce da un principio pedagogico ed epistemologico chiaro: **contrastare la delega cognitiva (*cognitive offloading*)** che sta avvenendo con l'uso acritico degli strumenti di *Generative Artificial Intelligence* (GenIA). In questa proposta, il sistema non impiega i *Large Language Model* (LLM) per risolvere i problemi o scrivere codice al posto dello studente, ma come **tutor socratici**:

1. **La mediazione maieutica è affidata all'LLM**: attraverso domande mirate, controesempi concreti ed elicitazione progressiva dei vincoli (livelli di aiuto graduati da L0 a L4), il tutor guida lo studente nell'individuare errori concettuali e nel motivare formalmente ogni scelta di modellizzazione.
2. **La verità tecnica risiede in controlli deterministici**: l'LLM non valuta la correttezza formale. La validazione sintattica, il calcolo della chiusura degli attributi, la verifica delle forme normali (2NF, 3NF, BCNF) e l'esecuzione di DDL e query SQL sono affidati a un runtime deterministico e a un esecutore isolato SQLite.
3. **La valutazione resta prerogativa esclusiva del docente**: il sistema non assegna voti, non stila punteggi né profila gli studenti. La verifica dell'apprendimento effettivo avviene tramite prove individuali in presenza senza ausilio del tutor (*prova di trasferimento*).

1.1 Piani di verifica della sperimentazione

La fattibilità e l'efficacia didattica della proposta sono da sperimentare su più piani (il dettaglio metodologico è approfondito nel documento di sperimentazione):

- **Piano del modello linguistico.** Non sappiamo se tra le proprietà emergenti dei LLM ci siano l'abilità maieutica e la capacità elicitativa che vorremmo. *Verifica:* golden set di casi noti (Sezione 5.5.3; Sezione 8.1).
- **Piano dell'infrastruttura di calcolo deterministica.** Un sistema che esegua i controlli e i processi richiesti è complesso: fattibilità e scalabilità non sono stimabili a priori, in assenza di prove empiriche. *Verifica:* test dei controlli deterministici (Sezione 5.5.1) e studio pilota (Sezione 8.1).
- **Piano dell'efficacia didattica.** Non è possibile stabilirla a priori: servono test diagnostici pre e post sperimentazione per misurare l'incremento delle prestazioni. *Verifica:* pre-test, post-test e prova di trasferimento (Sezione 8.1).
- **Piano della progressione didattica.** Il metodo e le fasi appaiono ben definiti per granularità e misurabilità, ma ciò va validato sperimentalmente: la suddivisione attuale potrebbe produrre un lavoro ingestibile per il docente o per lo studente, oppure risultare inefficace o inefficiente. *Verifica:* osservazione nello studio pilota (Sezione 8.1) e analisi dei risultati (Sezione 8.3).
- **Piano della sostenibilità per il docente.** Il tempo di preparazione delle tracce e di revisione dei risultati deve essere compatibile con classi numerose e tempi stretti (Sezione 4.8). *Verifica:* misura dei tempi nel pilota (Sezione 8.2).

1.2 Struttura del documento

La documentazione del progetto è strutturata nei seguenti moduli tematici:

- Stato dell'arte e inquadramento teorico: Intelligent Tutoring Systems, Socratic prompting, neuro-simbolico e quadro di ricerca.
- Quadro pedagogico: destinatari (ITE SIA, ITT Informatica, università), prerequisiti, obiettivi di apprendimento (O1–O8) e ruoli.
- Protocollo didattico: le nove fasi in cinque consegne, livelli di aiuto L0–L4, ciclo di revisione e sostenibilità docente.
- Architettura del sistema: orchestratore a stati finiti (FSM), sandbox SQLite isolata, hardware GPU on-premise, system prompt e GitLab CI.
- Specifiche dei formati e contratti dati: specifiche YAML validato per modelli concettuali e relazionali, formati di consegna e report.
- Quadro normativo, sicurezza e privacy: conformità GDPR, AI Act per sistemi scolastici, protezione da prompt injection e limiti noti.
- Protocollo di sperimentazione: disegno della ricerca, pre/post-test, pilota, metriche e criteri di successo.
- Bibliografia: riferimenti in formato BibTeX.

2. Stato dell'arte e inquadramento teorico

2.1 Introduzione e contesto

I modelli linguistici di grandi dimensioni (*Large Language Models*, LLM) hanno reso disponibili interfacce conversazionali capaci di produrre spiegazioni, esempi e codice in linguaggio naturale. Nella didattica dell'informatica, questa possibilità apre opportunità per sostenere l'esplorazione e il feedback, ma pone anche questioni relative all'affidabilità delle risposte, al ruolo attivo dello studente e alla valutazione dell'apprendimento [1], [2], [3]. Questi contributi discutono opportunità e criticità dell'IA generativa nell'istruzione e formazione informatica; non dimostrano, da soli, che un particolare modello o assetto didattico migliori gli apprendimenti in ogni contesto.

Nel progetto di basi di dati relazionali, l'attività non si esaurisce nella produzione di una query eseguibile. Richiede di interpretare requisiti, costruire un modello concettuale, tradurlo in uno schema relazionale, esplicitare chiavi e vincoli e formulare interrogazioni coerenti con il significato del problema. Il modello relazionale costituisce un riferimento teorico per la rappresentazione dei dati e delle relazioni tra essi [4], mentre il modello Entità/Associazioni è uno dei riferimenti classici per la progettazione concettuale [5]. La correttezza di una soluzione va quindi esaminata a più livelli: sintassi, proprietà formali dello schema, comportamento delle query e adeguatezza rispetto ai requisiti. Un controllo su un singolo insieme di dati, da solo, non prova la correttezza generale della soluzione.

Socratic-DB è una proposta di supporto alla progettazione e all'apprendimento delle basi di dati che combina tre componenti con responsabilità distinte:

1. **Controlli formalizzati**, per verificare le proprietà che possono essere espresse in modo deterministico, come vincoli strutturali, alcune proprietà dello schema e risultati dell'esecuzione SQL.
2. **Tutor conversazionale basato su LLM**, per porre domande, richiedere motivazioni, proporre controesempi e graduare gli aiuti, senza assumere che il modello segua sempre tali vincoli.
3. **Docente**, cui restano il giudizio sulle interpretazioni del dominio, la revisione delle diagnosi non formalizzabili e le decisioni valutative.

L'integrazione di questi componenti è una scelta progettuale, non una prova preventiva di correttezza complessiva o di efficacia didattica. Le domande di ricerca riguardano l'affidabilità dei controlli, la fedeltà del tutor al comportamento atteso, l'apprendimento autonomo e il trasferimento a problemi nuovi. Le ipotesi e il disegno di verifica sono descritti nel Capitolo 8..

2.2 Sistemi di Tutorato Intelligente e didattica dei database

I *Sistemi di Tutorato Intelligente* (*Intelligent Tutoring Systems*, ITS) costituiscono un precedente importante per il feedback automatizzato in domini tecnici. In questo ambito, il lavoro su SQL-Tutor di Mitrović illustra l'impiego del *Constraint-Based Modeling* (CBM): il sistema confronta la soluzione dello studente con vincoli disciplinari espliciti e può restituire feedback sulle violazioni individuate [6], [7]. Questo approccio offre un riferimento per rendere espliciti i criteri di controllo. Non implica, tuttavia, che ogni soluzione semanticamente valida possa essere riconosciuta automaticamente, né che lo stesso modello possa essere trasferito senza adattamenti a tutte le fasi della progettazione di una base di dati.

Socratic-DB condivide con gli approcci basati su vincoli l'idea di non affidare ogni giudizio tecnico alla generazione linguistica. La differenza proposta è l'ambito del percorso: oltre alla formulazione di query, il sistema considera analisi dei requisiti, modellazione concettuale, schema relazionale, implementazione e interrogazione. I controlli sono destinati a coprire proprietà esplicitamente formalizzate; la loro copertura effettiva deve essere misurata, e la correttezza semantica rispetto alla traccia non va considerata automaticamente risolta. Le checklist e i casi di prova richiedono definizione, revisione e manutenzione, mentre il docente deve poter riesaminare possibili falsi positivi o soluzioni alternative.

L'esecuzione di query su un database di prova permette di osservare risultati concreti e riproducibili. È però una forma di evidenza circoscritta: una query che produce il risultato atteso su un dataset potrebbe comportarsi diversamente su dati differenti o non rispettare tutti i requisiti. Per questo Socratic-DB

propone di affiancare l'esecuzione a controlli strutturali e a un dialogo che chieda allo studente di motivare le proprie scelte, senza trattare nessuna di queste fonti come sufficiente in isolamento.

2.2.1 Errori e misconcezioni nella formulazione di query SQL

La ricerca sull'apprendimento di SQL analizza difficoltà e misconcezioni che possono emergere mentre gli studenti formulano query. Miedema, Aivaloglou e Fletcher studiano il ragionamento dei principianti attraverso un protocollo *think-aloud* [8]; Taipalus, Siponen e Vartiainen esaminano errori e complicazioni ricorrenti nella formulazione di query [9]. Il lavoro di Miedema sulla formazione sui sistemi di dati offre ulteriore contesto per scomporre i concetti e le difficoltà del dominio [10]. Questi studi motivano l'attenzione a diagnosi più specifiche del solo messaggio d'errore del DBMS, ma i risultati ottenuti su compiti SQL non vanno trasferiti automaticamente alla modellazione concettuale o alla progettazione relazionale.

Ponzini, Guerrini e Catania propongono una revisione delle tassonomie degli errori SQL basata sulla rilevazione automatizzata [11]. Questo filone è pertinente alla progettazione di report e categorie d'errore: una classificazione esplicita può rendere le diagnosi più confrontabili e consentire di studiare quali difficoltà ricorrono. La classificazione automatica, tuttavia, non garantisce da sola che una diagnosi sia corretta in ogni caso, che distingua sempre un errore da una soluzione alternativa valida o che il feedback migliori l'apprendimento.

La proposta di Socratic-DB è di usare categorie di errore e controlli formalizzati come supporto al dialogo, non come sostituti del ragionamento dello studente o della revisione del docente. La tassonomia dovrà essere adattata al compito specifico, soprattutto quando il problema riguarda l'interpretazione dei requisiti e non soltanto la sintassi o la semantica di una query.

2.2.2 Modelli mentali e spiegazioni dei sistemi di dati

Miedema et al. discutono il concetto di *notional machine* per spiegare concetti dei sistemi di dati nell'insegnamento dell'informatica [12]. Questo lavoro è pertinente alla scelta di non limitare il feedback alla forma testuale di una query: l'apprendimento richiede anche di costruire e verificare un modello di ciò che la query o lo schema rappresentano e producono. In Socratic-DB, una possibile traduzione progettuale consiste nel chiedere allo studente di prevedere il comportamento di una query, motivare una scelta di schema e poi confrontare la previsione con l'esecuzione o con un controesempio. L'utilità di questa sequenza per il sistema proposto resta un'ipotesi da verificare.

2.2.3 Generazione di esercizi SQL con LLM e validazione

I lavori sulla generazione personalizzata di esercizi SQL con LLM e validazione basata su vincoli [13], [14] affrontano un problema vicino, ma distinto, da quello del tutoraggio dialogico: produrre esercizi e controllarne determinate proprietà non equivale a guidare uno studente nella modellazione, nella revisione di una soluzione o nella giustificazione delle scelte. Per Socratic-DB, la lezione progettuale è circoscritta: quando si usa un modello generativo per produrre materiale o spiegazioni, è opportuno esplicitare quali proprietà sono validate e quali rimangono da revisionare.

Un esempio di attività didattica in questo ambito è il laboratorio «Apprendimento di SQL basato su errori frequenti» di Guerrini e Ponzini, pubblicato negli atti ITADINFO 2026 [15]. Il laboratorio propone attività per far emergere e classificare errori, discutere possibili misconcezioni e usare IA generativa per spiegazioni ed esercizi mirati. Il contributo richiama anche LensSQL come tutor e RelaX come strumento per l'algebra relazionale [11], [13]. Si tratta di un antecedente per l'uso didattico dell'analisi degli errori e di strumenti di supporto; la descrizione dell'attività non va interpretata come prova sperimentale dell'efficacia di Socratic-DB o di un tutor LLM nella progettazione completa di basi di dati.

2.3 LLM nell'istruzione informatica e tutoraggio dialogico

2.3.1 Opportunità e rischi dell'uso generativo

Gli studi sull'IA generativa nell'istruzione alla programmazione discutono opportunità e criticità degli strumenti capaci di produrre direttamente codice o risposte [1], [2], [3]. Il possibile effetto sull'apprendimento dipende dal compito, dal livello dello studente, dal tipo di assistenza e da come viene organizzata l'attività; non è corretto assumere un effetto uniforme in tutti i contesti.

Quando un sistema fornisce direttamente una soluzione completa, esiste il rischio che lo studente accetti il risultato senza saperlo spiegare, verificare o trasferire a un problema diverso. Per Socratic-DB questo è un rischio progettuale da affrontare, non un effetto dato per dimostrato. Per valutarlo si propone di osservare non solo il prodotto finale, ma anche le motivazioni espresse, le revisioni effettuate e la capacità di affrontare una traccia nuova senza assistenza. La prova di trasferimento e il progetto di sperimentazione sono descritti nel Sezione 8.1.

2.3.2 Tutoraggio dialogico, scaffolding e limiti degli LLM

Bloom discute la differenza tra tutoraggio individuale e istruzione di gruppo e il problema di trasferire i benefici del primo a contesti più ampi [16]. Vygotskij costituisce un riferimento teorico per il ruolo dell'interazione e della mediazione nell'apprendimento e per il concetto di Zona di Sviluppo Prossimale [17]. Questi riferimenti forniscono un quadro pedagogico generale; non dimostrano direttamente che un LLM che pone domande socratiche produca gli stessi effetti di un tutor umano, né che il dialogo, da solo, garantisca l'apprendimento.

Macina et al. studiano opportunità e limiti dei tutor dialogici neurali in compiti di apprendimento linguistico. Gli autori rilevano che le prestazioni sono migliori in scenari didattici vincolati e più deboli in scenari meno circoscritti, e riportano errori di ragionamento in una quota significativa delle conversazioni valutate [18]. Il risultato è rilevante come avvertimento metodologico: un LLM non deve essere considerato un tutor affidabile soltanto perché produce un dialogo plausibile. Il dominio studiato non coincide con la progettazione di basi di dati, quindi le conclusioni non vanno trasferite direttamente a Socratic-DB.

Socratic-DB traduce questi riferimenti in ipotesi di progetto: chiedere allo studente di esplicitare il ragionamento, formulare domande circoscritte, proporre controesempi e graduare il livello di aiuto, evitando per quanto possibile di fornire subito una soluzione completa. Queste regole devono essere valutate separatamente dalla correttezza tecnica: il tutor può non rispettare il comportamento atteso o formulare una diagnosi errata, e la presenza di un prompt non garantisce da sola la conformità. La fedeltà del dialogo, la qualità delle spiegazioni e l'effetto sugli apprendimenti sono pertanto oggetti di verifica distinti.

2.4 Architettura ibrida: controlli formali, esecuzione e dialogo

Socratic-DB propone un'architettura ibrida in cui il modello linguistico non è l'unica fonte di giudizio tecnico. I controlli deterministici sono usati solo per proprietà che siano state definite formalmente e implementate; l'esecuzione SQL permette di osservare il comportamento su un insieme di dati specifico; l'LLM sostiene il dialogo e il docente mantiene la responsabilità delle decisioni non automatizzabili. In questo testo, «ibrida» descrive la ripartizione progettuale dei compiti e non implica una garanzia generale di correttezza.

Limite da considerare	Conseguenza per l'attività	Scelta progettuale di Socratic-DB
Risposte linguistiche non affidabili	Un LLM può formulare affermazioni errate o non coerenti con la diagnosi tecnica.	Separare le diagnosi prodotte dai controlli formalizzati dal testo generato dal tutor; sottoporre a verifica le risposte del tutor.
Verifica limitata al comportamento osservato	Una query può produrre il risultato atteso sui dati di prova e fallire su altri dati o requisiti.	Eseguire query su dati di test progettati per coprire casi significativi e dichiarare esplicitamente l'ambito della verifica.
Comportamento dialogico non garantito	Il modello può fornire la soluzione troppo presto, essere incoerente o assecondare un'affermazione scorretta.	Definire livelli di aiuto e regole per il tutor, registrare le interazioni e valutare la conformità su casi di prova.
Controlli incompleti o troppo rigidi	Proprietà non formalizzate possono sfuggire; controlli eccessi-	Documentare l'ambito di ogni controllo, testare soluzioni al-

Limite da considerare	Conseguenza per l'attività	Scelta progettuale di Socratic-DB
	vamente restrittivi possono respingere soluzioni valide.	ternative e prevedere la revisione del docente.
Separazione tra prodotto e apprendimento	Una soluzione corretta non prova che lo studente sappia giustificarla o riprodurla autonomamente.	Valutare separatamente il prodotto, il processo di revisione e il trasferimento senza tutor.

La tabella descrive rischi da gestire e decisioni proposte, non risultati sperimentali già ottenuti. In particolare, l'esecuzione in SQLite può verificare che una query sia eseguibile e osservare il suo risultato su dati specifici, ma non dimostra da sola la correttezza rispetto a ogni interpretazione della traccia. Analogamente, i controlli sullo schema possono verificare solo le proprietà coperte dalla loro implementazione e non sostituiscono la revisione semantica del dominio.

2.5 Contributo della proposta e questioni aperte

Sulla base dei sistemi e dei filoni esaminati, il contributo progettuale che Socratic-DB intende esplorare è l'integrazione, in un unico percorso didattico, di:

- analisi dei requisiti e modellazione concettuale;
- traduzione nello schema relazionale e considerazione dei vincoli;
- generazione o verifica dell'implementazione e dei dati di prova;
- scrittura, esecuzione e revisione di query SQL;
- dialogo guidato da domande, motivazioni e controesempi;
- controlli deterministici per le sole proprietà formalizzate;
- valutazione dell'apprendimento separata dalla valutazione automatica del prodotto.

Non si sostiene che ciascuna componente sia nuova in assoluto. Il valore distintivo da verificare è la loro integrazione nel percorso completo, con una separazione esplicita tra controllo formale, mediazione linguistica e giudizio del docente. Rispetto ai sistemi centrati sulla query, Socratic-DB intende includere anche le fasi precedenti di analisi e progettazione; rispetto agli strumenti di generazione di esercizi, si concentra sul dialogo relativo al tentativo dello studente; rispetto a un LLM usato senza controlli esterni, prevede verifiche formalizzate ed esecuzione SQL; rispetto a un sistema interamente deterministico, usa un modello linguistico per interagire in linguaggio naturale. Queste differenze descrivono l'obiettivo architetturale e didattico, non una superiorità empiricamente dimostrata.

La sperimentazione dovrà distinguere almeno quattro aspetti:

- **Affidabilità tecnica:** quali errori rilevano i controlli, quali soluzioni alternative accettano e quali falsi positivi o negativi producono?
- **Fedeltà del tutor:** il modello rispetta i livelli di aiuto, pone domande pertinenti e non introduce diagnosi errate o soluzioni premature?
- **Apprendimento e trasferimento:** gli studenti migliorano nella progettazione autonoma e riescono a lavorare su una traccia nuova senza tutor?
- **Fattibilità:** i tempi, il carico di revisione per il docente e l'usabilità sono sostenibili nel contesto didattico previsto?

Queste domande richiedono evidenze diverse: un insieme di casi attesi e test negativi per i controlli; revisione umana delle risposte del tutor; prove iniziali e finali, possibilmente accompagnate da una prova di trasferimento; osservazioni di fattibilità. Un miglioramento osservato in un singolo pilota, soprattutto in assenza di un confronto adeguato, non basta a dimostrare che l'effetto sia causato dal tutor. Il protocollo è descritto nel Capitolo 8..

2.6 Quadro normativo e protezione dei dati

L'uso di strumenti di IA in contesti scolastici richiede una valutazione del trattamento dei dati personali e delle finalità effettive del sistema. Il GDPR stabilisce principi e obblighi relativi, tra l'altro, a liceità, limitazione delle finalità, minimizzazione, esattezza, conservazione e sicurezza dei dati [19]. Non è corretto ricavare dal Regolamento un divieto generale di usare qualsiasi fornitore cloud: la liceità e le condizioni

del trattamento dipendono dal caso concreto, dalla base giuridica, dai ruoli dei soggetti, dagli accordi applicabili, dalle misure di sicurezza e, quando pertinente, dalle regole sui trasferimenti di dati.

L'articolo 22 del GDPR riguarda determinate decisioni basate unicamente su trattamenti automatizzati che producano effetti giuridici o incidano in modo analogamente significativo sull'interessato, nei limiti e con le eccezioni previste dal Regolamento. Non è quindi corretto descriverlo come un divieto assoluto di ogni decisione automatizzata, né come un generico diritto alla spiegabilità applicabile indistintamente a ogni output di un LLM. In Socratic-DB, la scelta progettuale è che il sistema non assegni voti né sostituisca il docente nelle decisioni valutative; ciò va tradotto in procedure operative coerenti con il contesto istituzionale.

Il Regolamento europeo sull'Intelligenza Artificiale (AI Act) include nell'Allegato III, punto 3, determinate categorie di sistemi destinati all'istruzione e alla formazione professionale, tra cui alcuni sistemi destinati a valutare i risultati dell'apprendimento o a orientare il processo di apprendimento. La classificazione dipende dalla finalità prevista e dall'uso concreto del sistema; non è possibile stabilirla soltanto sulla base del nome del progetto o dell'affermazione che non assegna voti. La qualificazione applicabile a Socratic-DB dovrà essere verificata alla luce del testo normativo e delle disposizioni pertinenti con i referenti istituzionali competenti [20].

L'esecuzione on-premise, se adottata, può ridurre il trasferimento di dati a servizi esterni, ma non costituisce da sola una garanzia assoluta di conformità o riservatezza. Occorre considerare configurazione di rete, autenticazione, autorizzazioni, log, conservazione dei dati, aggiornamenti e procedure di gestione degli incidenti. La minimizzazione dei dati raccolti e l'uso di identificativi non direttamente riconducibili agli studenti, quando appropriato, sono misure da valutare nel quadro complessivo del trattamento, non sostituti degli altri obblighi applicabili.

2.7 Sintesi del posizionamento

La letteratura presa in esame offre riferimenti per il feedback basato su vincoli, l'analisi degli errori SQL, la spiegazione dei modelli mentali dei sistemi di dati e l'impiego di LLM nell'istruzione informatica. Non fornisce, nel suo insieme, una prova diretta dell'efficacia di Socratic-DB: i lavori citati riguardano sistemi, compiti e contesti differenti.

La differenza che la proposta intende mettere alla prova è l'integrazione di un percorso che attraversa le principali fasi della progettazione di basi di dati con un tutor dialogico, controlli deterministici circoscritti e un'esecuzione SQL reale, mantenendo espliciti i limiti di ciascun componente e la responsabilità del docente. L'eventuale valore aggiunto dovrà essere stabilito empiricamente rispetto a criteri dichiarati: affidabilità tecnica, qualità del dialogo, apprendimento autonomo, trasferimento e fattibilità didattica.

3. Specifica pedagogica

3.1 Destinatari

Studenti del secondo biennio e del quinto anno di un Istituto Tecnico Economico (ITE), indirizzo «Amministrazione, Finanza e Marketing», articolazione «Sistemi Informativi Aziendali» (SIA, codice ITSI), o di un Istituto Tecnico Tecnologico (ITT), indirizzo «Informatica e Telecomunicazioni», articolazione «Informatica» (codice ITIA), in classi da 5 a 20 persone, con lavoro individuale o a coppie.

Il sistema è progettato tenendo conto dei quadri di riferimento e dei nuclei fondanti della seconda prova scritta dell'Esame di Stato per le discipline d'indirizzo. I quadri di riferimento di ITIA e ITSI includono tra i nuclei la progettazione concettuale, logica e fisica di basi di dati e i linguaggi per basi di dati; la disciplina della seconda prova varia da un anno all'altro. Tali prove richiedono di ricavare le specifiche progettuali da una traccia reale, sviluppare la modellizzazione concettuale del problema, tradurla nel modello logico-relazionale, realizzare la base di dati e formulare le query di interrogazione in linguaggio SQL.

La sperimentazione può inoltre essere estesa e adattata come supporto all'auto-apprendimento guidato in un primo corso universitario di «Basi di Dati» (lauree triennali in Informatica o Ingegneria Informatica).

3.2 Prerequisiti

- **Matematica e logica di base**:
  - concetti elementari della teoria degli insiemi (rappresentazione, appartenenza, unione, intersezione, differenza/complementazione, prodotto cartesiano e concetto di relazione);
  - fondamenti di logica proposizionale e predicativa (valori di verità, connettivi logici `NOT`, `AND`, `OR`).
- **Concetti introduttivi di modellizzazione e basi di dati**:
  - comprensione intuitiva del modello concettuale (entità, associazione, attributo, cardinalità min/max e concetti base di gerarchia/specializzazione);
  - strutture del modello relazionale (concetti informali di tabella, riga/tupla, colonna/attributo, chiave primaria, chiave esterna e vincoli fondamentali di integrità).
- **Linguaggio SQL di base**:
  - familiarità elementare con la sintassi SQL per le interrogazioni semplici (clausole `SELECT`, `FROM`, `WHERE`); il consolidamento di interrogazioni complesse (`JOIN`, aggregazioni `GROUP BY` e sottoquery) è uno degli obiettivi formativi del percorso.
- **Competenze digitali e strumentali**:
  - uso base di un editor di testo o ambiente di sviluppo (IDE) e familiarità con i comandi fondamentali da terminale/CLI;
  - nozioni elementari di Git (`commit`, `push` e consegne tramite repository).

3.3 Obiettivi di apprendimento

Al termine lo studente è in grado di, **autonomamente e senza tutor**:

- **O1.** Ricavare da una traccia i dati, le operazioni e i vincoli (espliciti e impliciti), distinguendo ciò che è dichiarato da ciò che è assunto.
- **O2.** Costruire un modello concettuale Entità/Associazioni (*Entity/Relationship*, ER) con entità, attributi, associazioni, cardinalità minime e massime, e motivare ogni scelta.
- **O3.** Distinguere i vincoli rappresentabili nello schema da quelli che richiedono altri meccanismi (`CHECK`, `UNIQUE`, trigger, controllo applicativo).
- **O4.** Tradurre il modello concettuale in schema relazionale scegliendo strategie adeguate per gerarchie, associazioni 1:1 opzionali e obbligatorie, associazioni 1:N e associazioni N:M, trattando i valori nulli e verificando la normalizzazione (1NF, 2NF, 3NF; BCNF se giustificata).
- **O5.** Codificare lo schema usando i costrutti del *Data Definition Language* (DDL) di SQLite, con tipi e vincoli appropriati.

- **O6.** Costruire dati di test significativi, inclusi casi limite, nulli e assenza di corrispondenze, codificandoli con istruzioni del *Data Manipulation Language* (DML) di SQLite.
- **O7.** Scrivere query SQL corrette, verificarle su dati propri e motivarne la correttezza.
- **O8.** Rivedere la propria soluzione a partire da un feedback e giustificare la modifica. Apprendere un modello di sviluppo iterativo in cui la critica di ogni scelta conduce a un modello via via più completo, pertinente, minimale, intelligibile e coerente.

3.4 Ruoli

3.4.1 Docente

- Sceglie o inserisce la traccia (dal catalogo o da prove d'esame) e **valida** la checklist dei requisiti proposta dallo strumento d'autore. Facoltativamente fornisce uno schema canonico con dataset e query attese, per il controllo automatico dei risultati nella fase 8. Non scrive soluzioni alternative, dati di riferimento né test.
- Decide la sequenza, i tempi e i livelli di aiuto massimi consentiti per ciascun esercizio, e se abilitare la generazione automatica del DDL (fase 6) e dei dati di test (fase 7).
- Legge le consegne (commit e Merge Request) e il report compatto del processo; legge a campione motivazioni e dialoghi; commenta in modo umano.
- Esamina le segnalazioni di falsi positivi (soluzioni valide respinte) e decide se correggere la checklist o il controllo.
- **È l'unico a esprimere valutazioni.** Conduce la prova individuale di trasferimento.
- Revisiona periodicamente un campione di dialoghi per individuare errori del tutor LLM.

3.4.2 LLM (tutor)

- Compito principale: trasformare le *diagnosi strutturate* prodotte dai controlli deterministici in una risposta socratica, al livello di aiuto richiesto dallo stato del dialogo. Le diagnosi sono vincolanti: la LLM non le contraddice.
- Chiede allo studente di motivare le scelte e di riformulare.
- Non assegna punteggi, non dichiara «superato/non superato», non fornisce la soluzione completa, non esegue SQL.
- Dove non c'è diagnosi deterministica (decisioni progettuali e correttezza semantica di cardinalità e query), formula domande e controesempi, dichiarando che esistono più soluzioni accettabili.
- Si comporta come sistema esperto che valuta costantemente, ma fornisce il proprio punto di vista **solo su richiesta** (salvo nella fase 8, dove il parere sul risultato delle query fa parte del dialogo) e dichiarandolo *non vincolante* (a differenza delle diagnosi deterministiche), su:
  - l'eliminazione delle ambiguità terminologiche e semantiche;
  - la completezza rispetto alle specifiche e alla possibilità di formulare le interrogazioni richieste dal testo;
  - la correttezza del modello e dei vincoli, oppure della sintassi, rispetto ai costrutti del modello in uso;
  - la minimalità, intesa come non ridondanza nei modelli e come realizzazione di query efficienti che non richiedono elaborazioni non necessarie;
  - la pertinenza del lessico rispetto al testo e al dominio.
- Procede in modo maieutico ai livelli di aiuto L0–L2 e passa all'elicitazione ai livelli L3–L4.
- Nella fase 7, se il docente abilita la generazione automatica, propone i dati di test (istruzioni INSERT) coerenti con lo schema dello studente; l'output è sempre sottoposto a verifica deterministica.
- Nella fase 8 riceve dall'esecutore il risultato della query sui dati di test e formula un parere non vincolante sulla corrispondenza tra richiesta, query e risultato, trasformandolo in domande (righe attese, mancanti o duplicate); non scrive la query corretta.
- Se incerto, richiede di contattare il docente.

3.4.3 Sistema deterministico (orchestratore e controlli)

- Mantiene lo stato del percorso, valida i formati, esegue operazioni sullo schema (DDL), sui dati (DML) e sulle query (QL) in SQLite isolato e produce le diagnosi. Deriva dallo schema dello studente i test negativi e la copertura strutturale dei dati di test, verifica i dati proposti dalla LLM nella fase 7 e, se abilitato, genera il DDL dal modello relazionale consegnato. Passa al tutor il risultato delle query. Con lo schema canonico opzionale confronta i risultati con le query attese.

3.5 Verifica dell'apprendimento: prodotto, processo, trasferimento

La qualità della soluzione finale non dimostra che lo studente l'abbia compresa. Si raccolgono evidenze distinte:

Dimensione	Evidenza	Fonte
Qualità progettuale	Schema corretto, completo, pertinente, minimale	Controlli deterministici + lettura del docente
Ragionamento	Motivazioni di cardinalità, chiavi e vincoli	Campo `motivazione` nei file + dialogo (lettura del docente)
Revisione	Correzione autonoma dopo feedback	Storia dei commit e dei livelli di aiuto richiesti
Competenza SQL	Query eseguibili e corrette su dati significativi	Esecuzione automatica sui dati di test; parere non vincolante del tutor
Trasferimento	Soluzione di una traccia nuova, senza assistenza	Prova individuale in aula, valutata dal docente

Solo l'ultima riga, insieme alla lettura umana delle altre, concorre alla valutazione scolastica.

3.6 Rischi didattici e contromisure

- **Tutor come sistema di sblocco.** Chi deve raggiungere una soglia ottimizza il punteggio, non la comprensione. *Contromisura:* nessuna soglia; l'avanzamento è consigliato, non vincolato; i punti aperti restano visibili.
- **Delega del pensiero.** *Contromisura:* ogni consegna richiede una motivazione scritta dallo studente; richiesta di aiuto ai livelli alti registrata; prova finale senza tutor.
- **Giudizi non riproducibili.** *Contromisura:* ogni affermazione sulla correttezza proviene da un controllo deterministico ripetibile.
- **Fiducia eccessiva nel tutor.** *Contromisura:* il tutor dichiara quando non è certo e rimanda al docente.

4. Protocollo didattico

4.1 Il percorso: nove fasi in cinque consegne

Il percorso ha **nove fasi** di verifica, raggruppate in **cinque consegne** per contenere il carico di consegna e di tracciamento. Le prime quattro consegne si svolgono con il tutor; la quinta è individuale e senza tutor. Ogni consegna mantiene file e criteri distinti per ciascuna fase che contiene, così lo stato di padronanza resta per competenza. Il percorso distingue progettazione concettuale e logica, chiarisce i vincoli e introduce verifiche effettive dell'esecuzione SQL.

Consegna	Fasi	Contenuto	Automazione
C1. Requisiti e glossario	1–2	File di requisiti e glossario	Controllo deterministico di copertura rispetto alla checklist validata dal docente (termini e sinonimi attesi, definizioni non circolari); il tutor pone le domande
C2. Progettazione concettuale	3–4	Modello ER e vincoli del dominio, in sezioni distinte (ogni vincolo con tipo e formulazione)	Validatore e test di scenario; il tutor conduce l'elicitazione dei vincoli
C3. Progettazione logica e normalizzazione	5	Modello relazionale e meccanismo previsto per ciascun vincolo	Calcolo deterministico delle forme normali; il tutor discute le ridondanze
C4. Implementazione e interrogazione SQL	6–8	DDL, dati, query	Ambiente SQLite isolato con esecuzione immediata e diagnosi; i tre punti di verifica restano distinti
C5. Prova di trasferimento	9	Traccia nuova	Nessuna: prova individuale in aula, valutata dal docente

I criteri di ciascuna fase sono i seguenti:

#	Fase	Attività dello studente	Criteri verificabili (esempi)	Controllo
1	Requisiti	Elenca dati da memorizzare, operazioni richieste, vincoli espliciti e impliciti, ambiguità da chiarire	Copertura delle voci della checklist; vincoli impliciti individuati; ambiguità motivate	Checklist validata dal docente; domande del tutor
2	Nomi-verbi e glossario	Estrae concetti candidati e li definisce senza assumere che ogni	Ogni voce ha definizione non circolare; sinonimi/omonimi dichia-	Controllo di presenza e di circolarità semplice; tutor sul resto

#	Fase	Attività dello studente	Criteri verificabili (esempi)	Controllo
		nome sia entità e ogni verbo associazione	rati; motivazione della classificazione	
3	Modello concettuale ER	Entità, attributi, associazioni, cardinalità min/max, identificatori	Schema valido nel formato; ogni entità ha identificatore; ogni elemento tracciabile a un requisito; cardinalità motivate	Validatore (struttura); domande e controesempi del tutor sulle cardinalità, non vincolanti
4	Vincoli del dominio	Classifica i vincoli: rappresentati nello schema, CHECK/UNIQUE, inter-relazione o temporali (trigger/applicativo)	Ogni vincolo ha tipo, formulazione non ambigua e meccanismo previsto; i temporali definiscono intervallo, sovrapposizione e concorrenza	Checklist; tutor conduce l'elicitazione
5	Modello relazionale	Applica le regole di traduzione; PK, FK, nullabilità, N:M, gerarchie, 1:1; dipendenze funzionali	Nessuna FK orfana; PK in ogni tabella; N:M risolte; nulli coerenti con le partecipazioni minime; 1NF/2NF/3NF verificate sulle dipendenze dichiarate	Validatore + calcolo deterministico delle forme normali
6	Implementazione SQL (DDL)	Scrive il DDL SQLite con tipi e vincoli. Se il docente abilita la traduzione automatica, il DDL è generato deterministicamente dal modello relazionale consegnato e lo studente lo analizza e lo commenta	DDL eseguibile; struttura (`PRAGMA table_info`, `foreign_key_list`) coerente con lo schema relazionale; violazioni attese, derivate dallo schema, rifiutate	Esecuzione reale + test negativi derivati dallo schema
7	Popolamento e test	Scrive INSERT significativi: casi limite, nulli, assenze di corrispondenza. Se il docente abilita la generazione automatica, la LLM propone i dati di test coerenti con lo schema dello	Dati accettati dal DDL; copertura strutturale derivata dallo schema (righe minime per tabella, genitori senza figli, nulli nelle colonne nullabili, valori ripetuti); casi in-	Esecuzione reale + controlli di copertura derivati dallo schema

#	Fase	Attività dello studente	Criteri verificabili (esempi)	Controllo
		studente, che li verifica e li integra con casi propri	validi derivati dallo schema rifiutati	
8	Query e verifica	Scrive, esegue e motiva le query; esamina il risultato ottenuto sui dati di test	Query eseguibile; risultato esaminato e discusso; motivazione della scelta di JOIN/ aggregazione; con schema canonico, risultato uguale all'atteso	Esecuzione reale + parere non vincolante del tutor sul risultato; confronto con le query attese solo con schema canonico
9	Prova di trasferimento	Affronta una traccia nuova, individualmente e senza tutor	Rubrica del docente per O1–O7	Docente

4.1.1 Sulla generazione automatica

Per impostazione predefinita lo studente scrive DDL e INSERT: sono competenze da acquisire (O5, O6). Il docente può abilitare, per singolo esercizio, due scorciatoie:

- **DDL (fase 6):** generato da un *generatore deterministico* (non dalla LLM) a partire dallo schema relazionale, solo dopo che lo schema ha superato le verifiche indipendenti; la struttura ottenuta è poi confrontata con lo schema.
- **Dati di test (fase 7):** la LLM propone le istruzioni INSERT per lo schema dello studente (valori plausibili, senza dati personali reali). Lo script è accettato solo se supera una *verifica deterministica* derivata dallo schema: esecuzione senza errori sul DDL dello studente con chiavi esterne attive, copertura strutturale (righe minime per tabella, genitori senza figli, nulli, valori ripetuti), rifiuto dei casi invalidi. Se la verifica fallisce lo script non è consegnato: si rigenera (fino a un numero massimo di tentativi) oppure si segnala al docente.

Quando una scorciatoia è attiva, la competenza corrispondente non è esercitata in quella consegna e va verificata nella fase 9.

4.2 Errori oggettivi e decisioni progettuali

Il sistema distingue due classi di rilievi, con trattamento diverso:

- **Errori oggettivamente verificabili** nei file dello studente: formato non valido, FK verso tabella inesistente, tabella senza PK, N:M non risolta, DDL non eseguibile, violazione di un vincolo dichiarato dallo studente nel proprio schema, query non eseguibile, dipendenza funzionale dichiarata che viola la 3NF. Il tutor li segnala con certezza e li guida a correggerli.
- **Decisioni progettuali con più soluzioni corrette:** attributo o entità, strategia di traduzione di una gerarchia, accorpamento o tabella separata per una 1:1, uso di chiave surrogata. Il tutor non dichiara errore; chiede di motivare, mostra le conseguenze con un controesempio e accetta ogni soluzione coerente con i requisiti.
- **Correttezza semantica rispetto alla traccia** (cardinalità, entità, risultato delle query): non è verificata in modo deterministico, salvo l'uso opzionale di uno schema canonico con query attese. Il tutor formula un parere non vincolante, con domande e controesempi; l'accertamento finale spetta alla prova di trasferimento e al docente.

4.3 Livelli di aiuto progressivi

Livello	Forma dell'aiuto	Quando
L0	Solo esito dei controlli: area interessata e numero di problemi, senza dettagli	Primo tentativo
L1	Domanda mirata sul problema specifico	Dopo L0 se il problema persiste
L2	Controesempio dal dominio (dati concreti che violano l'intento)	Dopo L1 non risolutivo
L3	Spiegazione del principio applicabile (non della soluzione)	Dopo L2 o su richiesta
L4	Esempio svolto parziale su un caso analogo di altro dominio, seguito da un nuovo esercizio autonomo	Dopo L3 o su richiesta motivata

Regole: l'aiuto sale di un livello alla volta; lo studente può chiedere un livello superiore (la richiesta è registrata, senza penalità); il docente può fissare per esercizio il livello massimo; la soluzione completa non è mai fornita.

4.4 Ciclo di revisione

1. Lo studente propone una soluzione e ne spiega il motivo (campo `motivazione`).
2. I controlli deterministici producono le diagnosi; il tutor ne seleziona *una* e pone una domanda mirata.
3. Lo studente rivede la soluzione e giustifica la modifica nel messaggio di commit.
4. Il sistema riesegue i controlli sulla proprietà interessata, con criteri espliciti.
5. Se serve, il tutor passa al livello successivo; al livello L4 propone un esempio parziale e un nuovo esercizio autonomo.

4.5 Padronanza per competenza

Per ogni competenza (O1–O7) il sistema mostra, non un punteggio, ma lo stato di una checklist di criteri verificabili, con tre stati descrittivi:

- **Da lavorare:** uno o più criteri oggettivi non soddisfatti.
- **In sviluppo:** criteri oggettivi soddisfatti solo con aiuto di livello $\geq$ L3.
- **Acquisito sotto guida:** criteri soddisfatti con aiuto $\leq$ L2 e motivazioni presenti.

Questi stati sono *indicatori di processo per lo studente e per il docente*, non voti. L'acquisizione effettiva è accertata solo dalla fase 9. L'avanzamento è libero: il tutor può consigliare di tornare su un punto aperto, ma l'unico blocco è tecnico (non si può scrivere il DDL senza uno schema relazionale analizzabile sul piano sintattico dal parser e verificabile sul piano semantico dal validatore).

4.6 Precisazioni tecniche incorporate

- **Nomi e verbi:** euristica, non algoritmo. La classificazione dipende dal significato nel dominio; il tutor non la «corregge», chiede di motivarla.
- **Gerarchie:** l'eliminazione non è regola universale; si valutano accorpamento nella superclasse, nelle sottoclassi o tabelle separate, in base a copertura, disgiunzione e accessi.
- **Relazioni 1:1:** non si accorpano sempre. Si considerano partecipazione obbligatoria, nullabilità, UNIQUE sulla FK e dipendenze.
- **Normalizzazione:** 1NF, 2NF, 3NF sempre; BCNF solo se giustificata dalla traccia. La 4NF non è richiesta salvo previsione curricolare.
- **Valori nulli:** trattati già nella modellazione (partecipazione minima 0 o 1, attributi opzionali), poi tradotti in NOT NULL o in attributi nullabili.

- **Vincoli temporali:** es. «un medico non può visitare due pazienti nella stessa ora» richiede di definire intervallo (inizio/fine, aperto/chiuso), sovrapposizione e concorrenza. Un CHECK SQLite non può verificare vincoli su più righe: servono UNIQUE su slot discreti, trigger o controllo applicativo, e il test deve verificarlo.

4.7 Metriche separate

1. **Qualità del modello** (calcolata dai controlli): completezza rispetto ai requisiti del docente, correttezza dei controlli oggettivi, minimalità (elementi non tracciabili), pertinenza, forme normali raggiunte.
2. **Qualità della spiegazione** (letta dal docente su campione, con rubrica a 3 livelli): la motivazione cita requisiti, conseguenze, alternative scartate.
3. **Padronanza dello studente:** prova di trasferimento e capacità di revisione (dai commit).

Le tre misure non sono sommate in un punteggio unico.

4.8 Sostenibilità per il docente

Il carico di lavoro è un vincolo di progetto: un protocollo che richiede ore di preparazione per ogni traccia non è applicabile in classi numerose e con tempi stretti. L'automazione riduce solo la preparazione e la lettura; la valutazione resta esclusivamente umana.

4.8.1 Pacchetto minimo e pacchetto esteso

Il materiale richiesto al docente è ridotto all'essenziale; il resto si deriva dai file dello studente.

Elemento	Pacchetto minimo (predefinito)	Pacchetto esteso (opzionale)
Traccia	Sì	Sì
Checklist dei requisiti (5–10 voci), proposta dallo strumento d'autore e validata	Sì	Sì
Test negativi e copertura dei dati	Derivati dallo schema dello studente	Derivati dallo schema dello studente
Risultato delle query (fase 8)	Parere non vincolante del tutor su dati di test dello studente o proposti dalla LLM	Schema canonico, dataset e query attese: controllo deterministico dei risultati, riusabile dal catalogo
Soluzioni alternative e scenari di test	No	Solo nel golden set di ricerca

4.8.2 Strumento d'autore (AI dietro le quinte)

Il docente inserisce la traccia, senza soluzione. Lo strumento propone la checklist dei requisiti e, per il pacchetto esteso, schema canonico, dataset e query attese. Controlli deterministici ne verificano la coerenza interna. Il docente valida o affina il pacchetto, che viene poi congelato e versionato. Il materiale generato da una LLM non è affidabile per sé: la validazione umana è obbligatoria.

4.8.3 Catalogo di schemi-tipo

Domini ricorrenti (prenotazioni e ticketing, e-commerce, gestione ospedaliera, biblioteca e prestiti) con checklist, errori tipici e, per alcuni domini, schema canonico con dataset e query attese già pronti. Il docente sceglie il modello e lo adatta alla traccia.

4.8.4 Tracce d'esame come punto di partenza

Le prove degli Esami di Stato sono una fonte primaria di tracce. Prima dell'uso, verificare note legali e licenza del sito del Ministero, citare la fonte e rimandare al documento originale senza riprodurlo. Elenco

indicato dal proponente (i link non sono stati verificati uno per uno; il docente seleziona le prove con basi di dati):

Anno	Sessione	Codice	Link
2005	Prova nazionale	M733	PDF
2007	Prova nazionale	M733	PDF
2009	Prova nazionale	M733	PDF
2011	Prova nazionale	M733	PDF
2013	Prova nazionale	M733	PDF
2016	Ordinaria	M967	PDF
2016	Straordinaria	M967	PDF
2016	Suppletiva	M967	PDF
2016	Esempio di prova	M967	PDF
2018	Ordinaria	I044	PDF
2018	Suppletiva	I044	PDF
2018	Straordinaria	I044	PDF

Per la fase 9 non si usano tracce pubbliche di cui circolano soluzioni online (e che una LLM può aver visto in addestramento): si adattano (dominio, requisiti, vincoli) o se ne scrivono di inedite.

4.8.5 Report compatto

Per lo studente e per il docente: indicatori fattuali per fase (tentativi per tipo di diagnosi, livello di aiuto massimo richiesto), prodotti dai controlli deterministici e non da un giudizio della LLM. Esempio: «3 tentativi con diagnosi 3NF_VIOLATA nella fase 5». Nessun punteggio e nessuna inferenza sulla competenza; il report non è utilizzabile a fini valutativi.

4.8.6 Pre-screening delle motivazioni

Nel prototipo, regole deterministiche (lunghezza minima, assenza di riferimenti a elementi dello schema o ai requisiti, testo identico a quello di altri studenti) segnalano al docente i casi da leggere per primi. Il docente legge comunque un campione casuale, per non basarsi solo sulle segnalazioni. Un classificatore basato su modello è rinviato al sistema avanzato.

4.8.7 Ipotesi di carico (da misurare nel pilota)

Attività del docente	Ipotesi con strumento d'autore e catalogo	Misura nel pilota
Preparazione di una traccia	Scelta o inserimento della traccia e validazione della checklist (pacchetto minimo): 20–30 minuti	Minuti di validazione per traccia
Lettura dei risultati	Report compatto, casi segnalati e un campione di motivazioni e dialoghi	Minuti per classe e per consegna
Riferimento: preparazione manuale senza strumento	4–6 ore per traccia (stima del proponente)	Da rilevare su un campione di docenti

La stima non è un risultato: tempi di preparazione e di revisione sono misure del pilota.

5. Architettura del sistema e prototipo tecnico

5.1 Principi architetturali

- **Open source, on-premise, senza servizi esterni**: L'intera catena di elaborazione risiede sull'hardware dell'istituto scolastico per garantire la sovranità dei dati e la conformità al GDPR.
- **Agenti semplici con responsabilità singola**: Le decisioni sulla correttezza sintattica e formale sono rigorosamente deterministiche; l'LLM si occupa esclusivamente della mediazione maieutica e del dialogo.
- **Un solo formato testuale strutturato**: Formato YAML validato tramite JSON Schema per modelli concettuali e relazionali.
- **Consegna e feedback tramite repository Git**: Flussi di lavoro basati su commit incrementali e pipeline CI per tracciare il processo di revisione.

5.2 Architettura del prototipo

```
[Studente] --testo/YAML--> [Interfaccia unica: Open WebUI o CLI/FastAPI]
                                    |
                                    v
                        [Orchestratore (Python, FSM)]
              stato del percorso, livello di aiuto, log pseudonimi
                 |                    |                      |
                 v                    v                      v
     [Validatori YAML]     [Esecutore SQLite]      [Tutor LLM (1 modello)]
  schema, FK, PK, forme    isolato, in memoria,      riceve diagnosi, risultati
  normali (deterministici) dati di test              delle query e livello di aiuto
                           dello studente
                +---- diagnosi strutturate e risultati ----+
```

5.2.1 Flusso operativo

1. Lo studente consegna un file (tramite commit/MR o interfaccia web).
2. L'orchestratore valida la sintassi e la conformità allo schema atteso.
3. Vengono eseguiti i controlli deterministici (integrità referenziale, normalizzazione, esecuzione su SQLite isolato) e costruite le diagnosi strutturate.
4. Il tutor LLM riceve le sole diagnosi e il livello di aiuto corrente, trasformandole in una risposta socratica conforme alle regole pedagogiche.
5. L'orchestratore applica un filtro di sicurezza (controllo di non-fuga della soluzione) e restituisce la risposta allo studente.

5.3 Hardware e scelta del modello

Configurazione di riferimento per il server locale: **macchina Linux con 64 GB di RAM e GPU da 32 GB di VRAM**.

- **Runtime di inferenza:** Ollama (per semplicità di setup locale) oppure llama.cpp / vLLM (per massimo controllo, throughput e concorrenza); esposizione di API standard compatibile OpenAI su `localhost`.
- **Modello linguistico unico:** Un solo modello linguistico con provate capacità di coding e ragionamento in lingua italiana e licenza permissiva (es. Apache 2.0). Il modello deve essere quantizzato per risiedere

interamente nei 32 GB di VRAM (indicativamente modelli da ~30B parametri con quantizzazione a 4–5 bit e contesto utile di 16–32k token).
- **Candidati per il confronto sul golden set:**
  - `gpt-oss-20b` (Apache 2.0);
  - Modelli della famiglia Qwen 2.5 / 3 (30–32B parametri);
  - Modelli open weights equivalenti con forte supporto SQL e logica. La scelta finale è guidata empiricamente dai risultati sul golden set (Sezione 5.5), non dalla notorietà.
- **Parametri di inferenza:** Temperatura bassa ($T \approx 0.1 - 0.2$) e seed fisso nei test di conformità per massimizzare la riproducibilità.
- **Concorrenza:** Gestione di una coda FIFO con priorità singola GPU; con 5–20 studenti a ritmo di riflessione e digitazione, la latenza per turno di dialogo resta nell'ordine di pochi secondi.
- **Visione artificiale:** Esclusa dal prototipo iniziale; demandata al sistema avanzato (Sezione 5.9).

5.4 Esecutore SQLite isolato

Requisiti di isolamento e sicurezza per l'esecuzione di DDL, popolamento e query dello studente:

- **Isolamento a livello OS:** Processo separato in container Docker senza rete (`--network none`), utente senza privilegi (`nobody`/`runner`), filesystem montato in sola lettura ad eccezione di una directory `/tmp` volatile; in alternativa, `subprocess` vincolato con limiti rigidi di sistema (`rlimit` su CPU time, RSS memory e disk quota).
- **Database in memoria:** Utilizzo di database SQLite `:memory:` (o file temporaneo distrutto al termine dell'elaborazione); una nuova istanza pulita per ciascun tentativo.
- **Integrità referenziale attiva:** Esecuzione obbligatoria di `PRAGMA foreign_keys = ON;`.
- **Modalità sola lettura per la verifica delle query:** `PRAGMA query_only = ON;` o apertura della connessione in `mode=ro` durante la fase 8.
- **Autorizzatore SQLite attivo (`set_authorizer`):** Blocco categorico di istruzioni pericolose come `ATTACH DATABASE`, `DETACH DATABASE`, caricamento di estensioni native (`load_extension`), comandi di `PRAGMA` non autorizzati e operazioni di scrittura al di fuori delle fasi di DDL e popolamento.
- **Timeout e quote:** Limite di timeout stringente (es. massimo 2 secondi per query) e limite al numero di tuple restituite (max 100 righe) per prevenire blocchi da loop o prodotti cartesiani involontari.
- **Visibilità dei dati:** Lo studente esegue le proprie query esclusivamente sul proprio dataset di test. Se il docente attiva uno schema canonico con dataset di riferimento, quest'ultimo rimane completamente invisibile allo studente (il sistema notifica solo l'eventuale discrepanza o prima differenza riscontrata).

5.5 Test automatici del sistema

Il sistema integra tre famiglie di test automatici eseguibili in `pytest` e nella pipeline CI/CD:

5.5.1 Test dei controlli deterministici

- **Validatore sintattico e semantico:** File YAML validi e casi di test negativi per ciascuna regola (FK orfane, PK mancanti, associazioni N:M non tradotte in tabelle ponte).
- **Calcolo delle forme normali:** Test con insiemi di dipendenze funzionali note per verificare il calcolo algebrico della chiusura degli attributi e la rilevazione di violazioni di 2NF, 3NF e BCNF.
- **Robustezza del sandbox:** Verifica che tentativi di attacco (injection di `ATTACH`, loop di ricorsione CTE, esaurimento memoria) vengano bloccati dall'autorizzatore e terminati per timeout.
- **Coerenza DDL ↔ Modello relazionale:** Verifica deterministica che le tabelle, colonne, indici e vincoli creati in SQLite (`PRAGMA table_info`, `PRAGMA foreign_key_list`) coincidano perfettamente con lo schema relazionale dichiarato nello YAML.
- **Derivazione automatica dei test negativi:** Generazione deterministica di casi di test che violano vincoli di integrità dichiarati (duplicazione PK, violazione NOT NULL, violazione CHECK).

5.5.2 Test sugli esercizi (Golden set di ricerca e schema canonico)

- **Checklist dei requisiti:** Verifica che la copertura semantica dei termini sia rilevata correttamente sia su soluzioni esaustive sia su consegne parziali.

- **Golden set di ricerca:** Insieme calibrato di tracce con soluzioni multiple valide e soluzioni errate ottenute per mutazione deterministica (es. rimozione di una chiave esterna, cardinalità invertita, vincolo omesso), ciascuna con la corrispondente diagnosi attesa.
- **Schema canonico:** Esecuzione delle query attese sul dataset di riferimento con confronto tra insiemi di risultati.

5.5.3 Test del tutor LLM (Golden set)

- **Casi di test:** Almeno 30 casi per ciascuna fase (soluzione corretta, soluzione con errore tipico, soluzione alternativa valida e non convenzionale).
- **Assenza di contraddizioni:** Il tutor non deve mai smentire una diagnosi deterministica né segnalare errori fittizi su varianti valide.
- **Aderenza al livello di aiuto:** A livello L1 il messaggio deve contenere solo domande; controesempi sono ammessi solo da L2; spiegazioni del principio solo da L3; nessun codice risolutivo completo a nessun livello.
- **Filtro di non-fuga della soluzione:** Verifica automatica che la risposta del tutor non contenga blocchi di codice completi o query risolutive; in caso di violazione, la risposta è rigettata e rigenerata con un prompt di ammonimento.
- **Valutazione semantica su query (fase 8):** Misura dell'accuratezza del parere non vincolante del tutor rispetto a valutatori umani su un corpus di query con errori tipici (filtro errato, JOIN incompleta, raggruppamento assente).
- **Validazione umana e accordo inter-valutatore:** Revisione in cieco su un campione di risposte da parte di docenti con calcolo dell'indice Kappa di Cohen.

5.5.4 Verifica deterministica dei dati di test generati dalla LLM (fase 7, se abilitata)

Quando il docente abilita la generazione assistita dei dati: - Lo script SQL generato deve essere composto unicamente da istruzioni `INSERT`. - Esecuzione obbligatoria senza errori sul DDL dello studente con `PRAGMA foreign_keys = ON;`. - Verifica della copertura strutturale minima derivata dallo schema: numero minimo di righe per tabella, presenza di almeno un'istanza genitore priva di figli, presenza di valori nulli nelle colonne opzionali, valori ripetuti per consentire test di aggregazione (`GROUP BY` e `COUNT`). - I casi invalidi generati per test negativo devono fallire l'inserimento.

5.6 System prompt del tutor (Versione per il prototipo)

```
Sei DB-Socrates, tutor di Basi di Dati. Ricevi in ingresso: (1) la fase corrente,
(2) il file consegnato dallo studente, (3) un elenco di DIAGNOSI prodotte da
controlli automatici, (4) il LIVELLO DI AIUTO consentito (L0-L4),
(5) le eventuali SCORCIATOIE abilitate dal docente (DDL, DATI),
(6) in fase 8, il RISULTATO della query sui dati di test.

Regole:
- Le diagnosi sono la verita tecnica: non contraddirle e non aggiungerne.
- Scegli UNA diagnosi alla volta e rispondi solo al livello di aiuto indicato:
  L0-L2 maieutica (esito, domanda, controesempio); L3-L4 elicitazione
  (principio, esempio parziale su un altro dominio).
- Non fornire mai schemi, DDL o query risolutive. Non assegnare punteggi.
- Per le decisioni progettuali segnala che esistono piu soluzioni valide e chiedi
  di motivare la scelta.
- Esprimi un parere su completezza, minimalita, pertinenza o lessico solo se lo
  studente lo chiede, dichiarando che non e vincolante.
- Fase 8: confronta la richiesta, la query e il RISULTATO ricevuto; esprimi un
  parere non vincolante e trasformalo in domande (righe attese, mancanti o
  duplicate). Se i dati di test non permettono di distinguere una query
  corretta da una sbagliata, dillo e chiedi allo studente di aggiungere casi.
- Chiedi sempre allo studente di giustificare la modifica che propone.
- Se non sei sicuro, dillo e rimanda al docente.
```

```
- Se la scorciatoia DATI e attiva (fase 7): proponi INSERT coerenti con lo schema
  dello studente (valori plausibili, senza dati personali reali, con genitori
  senza figli, nulli e valori ripetuti); nessun altro codice.
- Rispondi in italiano, in massimo 150 parole (esclusi gli INSERT generati).
```

5.7 GitLab / Forgejo come piattaforma di consegna e revisione

Infrastruttura basata su istanza on-premise di **GitLab Community Edition** (o **Forgejo / Gitea**) affiancata da un runner Docker privo di connessione Internet.

- **Repository modello:** Predisposto dal docente per ogni esercitazione contenente: traccia, template vuoti dei file di consegna, schema JSON di validazione e file `.gitlab-ci.yml`.
- **Workflow studente:** Ogni studente o coppia lavora in un fork privato (o branch isolato) identificato da uno pseudonimo (es. `s017`).
- **File di consegna:**

```
01_requisiti.md
02_glossario.md
03_concettuale.yaml
04_vincoli.md
05_relazionale.yaml
06_schema.sql
07_dati.sql
08_query.sql
motivazioni.md
```

- **Commit motivati:** Ogni modifica deve essere accompagnata da un messaggio di commit esplicativo che motivi il perché della variazione; la cronologia Git costituisce la prova primaria della capacità di revisione autonoma.
- **Merge Request e Pipeline CI:** L'apertura della Merge Request per ciascuna consegna scatena la pipeline di validazione:

```
stages: [valida, esegui]

valida:
  stage: valida
  script:
    - python -m socratic.validate 03_concettuale.yaml 05_relazionale.yaml

esegui:
  stage: esegui
  script:
      - python -m socratic.run --ddl 06_schema.sql --dati 07_dati.sql --query
08_query.sql --test tests/
  allow_failure: true
```

- **Review umana del docente:** Il docente può intervenire direttamente nella discussione della Merge Request commentando a campione gli elaborati. Tutti i commenti automatici del sistema riportano chiaramente la dicitura: *«Verifica tecnica automatica, priva di valore valutativo»*.

5.8 Strumento d'autore per il docente

Agente specializzato a compito unico per accelerare la preparazione dei materiali didattici senza trattare dati personali degli studenti:

1. **Input:** Inserimento del testo della traccia e selezione opzionale di uno schema-tipo dal catalogo di dominio.

2. **Generazione della bozza:** L'LLM propone la checklist dei requisiti (5–10 punti chiave); per il pacchetto esteso genera lo schema canonico, il dataset e le query attese per la fase 8.
3. **Controllo automatico di coerenza:** I validatori verificano la corrispondenza lessicale tra traccia e checklist e l'eseguibilità dello schema canonico.
4. **Validazione umana obbligatoria:** Il docente esamina la proposta tramite interfaccia visuale o Merge Request Git, apporta eventuali modifiche e approva il pacchetto, che viene congelato con un tag di versione immutabile.
5. **Ciclo di miglioramento continuo:** Eventuali falsi positivi segnalati dagli studenti consentono al docente di raffinare la checklist con una nuova versione del tag.

5.9 Ipotesi di sistema avanzato (Post-sperimentazione)

Funzionalità previste per successive evoluzioni della piattaforma: - **Visione artificiale per schemi manoscritti:** Modello multimodale (VLM) per la trascrizione automatica di diagrammi concettuali disegnati a mano libera su carta; la trascrizione produce un'ipotesi YAML che lo studente corregge e valida prima di avviare il dialogo. - **Architettura multi-agente specializzata:** Disaccoppiamento tra agente trascrittore, agente per l'analisi di vincoli complessi, generatore di controesempi e sintetizzatore di report per il docente, coordinati da una FSM centrale. - **Supporto multi-notazione:** Convertitori bidirezionali tra formato YAML e formati grafici o testuali (Mermaid, PlantUML, diagrammi relazionali Chen/Crow's Foot). - **RAG locale:** Retrieval-Augmented Generation su slide di lezione, testi di riferimento e dispense del docente. - **Cruscotto analitico:** Dashboard con indicatori aggregati di classe per identificare i concetti critici (es. frequenza di violazioni della 3NF, tempo medio di risoluzione delle chiavi esterne).

6. Specifiche dei formati e contratti dati

Per garantire interoperabilità, elaborazione automatica deterministica e feedback preciso da parte dell'LLM, il sistema adotta formati testuali standard e rigorosamente tipizzati.

6.1 Specifiche YAML per i modelli

Tutti i modelli prodotti dagli studenti adottano la sintassi **YAML**, validata a tempo di commit o upload mediante **JSON Schema** o schemi **Pydantic**. I diagrammi visuali (es. Mermaid) sono generati dal sistema per visualizzazione a partire dal file YAML e non costituiscono l'input primario.

6.1.1 Modello Concettuale (`tipo: concettuale`)

Il modello concettuale formalizza entità, attributi, identificatori univoci, associazioni, cardinalità minime/massime e vincoli di dominio. Ogni entità e associazione include un campo `motivazione` in cui lo studente giustifica la scelta rispetto al testo del problema.

```yaml
versione: 1
tipo: concettuale

entita:
  - nome: Paziente
    attributi:
      - {nome: cf, tipo: stringa, identificatore: true}
      - {nome: cognome, tipo: stringa, opzionale: false}
      - {nome: nome, tipo: stringa, opzionale: false}
      - {nome: telefono, tipo: stringa, opzionale: true}
    motivazione: "Il codice fiscale identifica univocamente la persona nel dominio."

  - nome: Medico
    attributi:
      - {nome: matricola, tipo: stringa, identificatore: true}
      - {nome: specializzazione, tipo: stringa, opzionale: false}
    motivazione: "La matricola ospedaliera è l'identificatore interno del medico."

associazioni:
  - nome: Visita
    partecipanti:
      - {entita: Paziente, card: "0..N"}
      - {entita: Medico, card: "0..N"}
    attributi:
      - {nome: inizio, tipo: datetime}
      - {nome: durata_min, tipo: intero}
     motivazione: "Una visita registra data, ora e durata dell'incontro tra paziente
e medico."

vincoli:
  - id: V1
       testo: "Un medico non può effettuare due visite con intervalli temporali
sovrapposti."
    classe: temporale
    meccanismo: trigger
  - id: V2
    testo: "La durata della visita deve essere strettamente positiva."
```

```
    classe: attributo
    meccanismo: check
```

6.1.1.1 Regole di validazione semantica concettuale

1. **Unicità dei nomi**: Nessuna entità o associazione può avere nomi duplicati.
2. **Identificatore presente**: Ogni entità deve possedere almeno un attributo con `identificatore: true` (o identificatore esterno dichiarato).
3. **Cardinalità valide**: I valori di `card` ammessi sono `'0..1'`, `'1..1'`, `'0..N'`, `'1..N'` (o notazioni equivalenti min/max).
4. **Tracciabilità**: Ogni entità deve essere richiamata in almeno una associazione o giustificata esplicitamente.

6.1.2 Modello Relazionale (`tipo: relazionale`)

Il modello relazionale definisce tabelle, tipi di dato SQL, nullabilità, chiavi primarie, chiavi esterne, vincoli di unicità, check e **dipendenze funzionali dichiarate**. La presenza esplicita delle dipendenze rende possibile il calcolo deterministico delle chiavi candidate e la verifica formale di 2NF, 3NF e BCNF.

```
versione: 1
tipo: relazionale

tabelle:
  - nome: Paziente
    colonne:
      - {nome: cf, tipo: TEXT, nullo: false}
      - {nome: cognome, tipo: TEXT, nullo: false}
      - {nome: nome, tipo: TEXT, nullo: false}
      - {nome: telefono, tipo: TEXT, nullo: true}
    pk: [cf]
    motivazione: "Tabella anagrafica pazienti con PK codice fiscale."

  - nome: Medico
    colonne:
      - {nome: matricola, tipo: TEXT, nullo: false}
      - {nome: specializzazione, tipo: TEXT, nullo: false}
    pk: [matricola]
    motivazione: "Tabella anagrafica medici con PK matricola."

  - nome: Visita
    colonne:
      - {nome: id, tipo: INTEGER, nullo: false}
      - {nome: cf_paziente, tipo: TEXT, nullo: false}
      - {nome: matricola_medico, tipo: TEXT, nullo: false}
      - {nome: inizio, tipo: TEXT, nullo: false}
      - {nome: durata_min, tipo: INTEGER, nullo: false}
    pk: [id]
    fk:
      - colonne: [cf_paziente]
        riferisce: "Paziente(cf)"
        on_delete: RESTRICT
        on_update: CASCADE
      - colonne: [matricola_medico]
        riferisce: "Medico(matricola)"
        on_delete: RESTRICT
        on_update: CASCADE
    unique:
      - [matricola_medico, inizio]
    check:
```

```
      - "durata_min > 0"
    dipendenze:
      - "id -> cf_paziente, matricola_medico, inizio, durata_min"
      - "matricola_medico, inizio -> id, cf_paziente, durata_min"
     motivazione: "Chiave surrogata id per gestire visite multiple tra gli stessi
soggetti."
```

6.1.2.1 Regole di calcolo deterministico

- **Chiusura degli attributi (X^+)**: Calcolo automatico della chiusura degli insiemi di attributi rispetto alle dipendenze dichiarate.
- **Rilevazione chiavi candidate**: Verifica che la chiave primaria scelta (`pk`) sia effettivamente una superchiave minimale.
- **Forme normali**:
  - *1NF*: Valori atomici e presenza di chiave primaria.
  - *2NF*: Assenza di dipendenze parziali di attributi non primi dalla chiave primaria.
  - *3NF*: Assenza di dipendenze transitive verso attributi non primi.
  - *BCNF*: Ogni determinante nelle dipendenze è una superchiave.

6.2 Struttura dei file di consegna dello studente

Il repository o spazio di lavoro dello studente si compone di file standardizzati:

File	Formato	Contenuto
`01_requisiti.md`	Markdown	Elenco di dati da memorizzare, operazioni richieste, vincoli espliciti/impliciti e ambiguità rilevate.
`02_glossario.md`	Markdown	Glossario concetti estratti (nomi/verbi), definizioni formali non circolari, sinonimi e omonimi.
`03_concettuale.yaml`	YAML	Modello concettuale ER secondo la specifica Sezione 6.1.1.
`04_vincoli.md`	Markdown	Tabella di classificazione dei vincoli di business e meccanismo di enforcement scelto (schema, check, trigger, app).
`05_relazionale.yaml`	YAML	Modello relazionale secondo la specifica Sezione 6.1.2.
`06_schema.sql`	SQL (SQLite)	Istruzioni DDL (`CREATE TABLE`, vincoli `CHECK`, `UNIQUE`, indici).
`07_dati.sql`	SQL (SQLite)	Istruzioni DML (`INSERT`) con dataset significativo e casi limite.
`08_query.sql`	SQL (SQLite)	Query `SELECT` con commento di motivazione della formulazione logica.
`motivazioni.md`	Markdown	Registro narrativo delle decisioni di riprogettazione adottate a seguito dei feedback.

6.3 Formato delle diagnosi strutturate

Quando l'orchestratore esegue i validatori deterministici, genera un payload JSON standardizzato che viene fornito al modello linguistico tutor:

```
{
  "consegna_id": "C3",
  "fase": 5,
  "livello_aiuto": "L1",
  "errori_rilevati": [
    {
      "codice": "3NF_VIOLATA",
      "gravita": "errore_oggettivo",
      "tabella": "Ordine",
     "dettaglio": "L'attributo 'citta_cliente' dipende transitivamente da 'cliente_id'
e non direttamente dalla PK 'ordine_id'.",
      "dipendenza_violante": "cliente_id -> citta_cliente"
    },
    {
      "codice": "FK_ORFANA",
      "gravita": "errore_oggettivo",
      "tabella": "Dettaglio",
     "dettaglio": "La chiave esterna 'prodotto_id' riferisce una colonna non presente
nella tabella Prodotti."
    }
  ],
  "metriche": {
    "completezza_requisiti": 0.85,
    "copertura_fk": 0.90,
    "forme_normali": {
      "Ordine": "2NF",
      "Cliente": "3NF",
      "Prodotto": "3NF"
    }
  }
}
```

Il tutor riceve questo payload nel prompt di sistema e seleziona **una sola diagnosi alla volta** per non sovraccaricare cognitivamente lo studente.

6.4 Formato del report compatto di processo

Per consentire al docente una consultazione rapida del percorso senza applicare automatismi valutativi, il sistema genera un report sintetico per studente:

```
======================================================================
REPORT PROCESSO: Studente s042 - Traccia: Gestione Ospedaliera (M733)
======================================================================
Fase 1 (Requisiti):           4 tentativi | Livello aiuto max: L1
Fase 2 (Glossario):           2 tentativi | Livello aiuto max: L0
Fase 3 (Concettuale ER):      6 tentativi | Livello aiuto max: L2
   - Diagnosi ricorrente:     CARDINALITA_INVERTITA (2 volte)
Fase 4 (Vincoli):             3 tentativi | Livello aiuto max: L2
Fase 5 (Relazionale):         5 tentativi | Livello aiuto max: L3
   - Diagnosi ricorrente:     3NF_VIOLATA (3 volte)
Fase 6 (DDL SQLite):          2 tentativi | Livello aiuto max: L1
Fase 7 (Popolamento/Test):    3 tentativi | Livello aiuto max: L1
Fase 8 (Query SQL):           4 tentativi | Livello aiuto max: L2

Note e flag di attenzione:
- Superamento di L2 nella fase 5 (richiesto supporto di principio su normalizzazione).
```

```
- Motivazioni sintetiche segnalate dal pre-screening per la fase 3 (righe < 3).
========================================================================
```

7. Quadro normativo, sicurezza e privacy

Quanto segue costituisce una traccia operativa di conformità normativa e di sicurezza da validare con il Responsabile della Protezione dei Dati (DPO) dell'istituzione scolastica e con il Dirigente Scolastico prima dell'avvio della sperimentazione; non costituisce parere legale formale.

7.1 Trattamento dei dati personali (GDPR)

La tabella seguente sintetizza le scelte di conformità al Regolamento Generale sulla Protezione dei Dati (GDPR - Regolamento UE 2016/679):

Aspetto	Scelta e misura operativa
Titolare del trattamento	L'istituzione scolastica (o universitaria) in cui si svolge la sperimentazione.
Finalità	Supporto didattico formativo curricolare; eventuale ricerca didattica accademica in forma rigorosamente aggregata e anonima.
Base giuridica	Esecuzione di un compito di interesse pubblico o connesso all'esercizio di pubblici poteri di cui è investito il titolare (art. 6.1.e GDPR); per la ricerca didattica, specifica informativa e acquisizione del consenso o altra base identificata con il DPO.
Tutela dei minori	Studenti in prevalenza minorenni (circa 14–19 anni): erogazione di informativa trasparente e accessibile a studenti ed esercenti la responsabilità genitoriale.
Dati trattati	Elaborati didattici (file YAML, query SQL, testi di motivazione), log delle interazioni e delle diagnosi; **divieto categorico di inserire dati personali reali** nelle tracce e nei dataset di prova.
Minimizzazione e pseudonimizzazione	Adozione di identificatori pseudonimi casuali (es. `s017`); nessun nome o dato anagrafico inserito nei prompt; per le immagini di schemi manoscritti (sistema avanzato), distruzione del file immagine immediatamente dopo la trascrizione testuale.
Conservazione	Conservazione dei log tecnici e dei dialoghi limitata alla durata dell'anno scolastico, con successiva cancellazione sicura o anonimizzazione irreversibile; le pubblicazioni di ricerca utilizzano solo dati aggregati.
Ruoli e accessi	Studente: accesso esclusivo ai propri dati. Docente: accesso ai dati e ai report della propria classe. Amministratore di sistema: accesso per sole finalità di manutenzione tecnica.
Destinatari terzi	**Nessun destinatario esterno:** modelli di inferenza, banche dati ed esecutori operano intera-

Aspetto	Scelta e misura operativa
	mente in rete locale (*on-premise*). Nessun dato transita verso API cloud commerciali.
Diritti degli interessati	Esercizio dei diritti di accesso, rettifica, cancellazione e limitazione garantito e gestito direttamente tramite la segreteria dell'istituto.

7.1.1 Adempimenti documentali

Prima dell'inizio delle attività devono essere predisposti: 1. Informativa sul trattamento dei dati per studenti e famiglie; 2. Aggiornamento del Registro dei Trattamenti dell'istituto; 3. Valutazione d'Impatto sulla Protezione dei Dati (DPIA) per l'introduzione di strumenti di intelligenza artificiale con minori; 4. Linee guida per gli studenti per l'uso esclusivo di dati sintetici o di fantasia nei test SQL.

7.2 Misure di sicurezza tecniche e organizzative

- **Isolamento di rete:** I server di inferenza (Ollama/vLLM) e i container di esecuzione risiedono nella sottorete dell'istituto priva di connessioni in uscita verso Internet; gli aggiornamenti di sistema e dei pacchetti Python/OS sono effettuati periodicamente tramite mirror o repository controllati.
- **Autenticazione e controllo degli accessi:** Integrazione con i sistemi di identità scolastici (SSO, LDAP/Active Directory o credenziali pseudonime gestite internamente); cifratura a riposo dei dischi server (`LUKS`) e dei backup.
- **Isolamento dell'esecutore SQL:** I file SQL consegnati sono trattati come non fidati. L'esecuzione avviene in container monouso senza rete o processi vincolati con `rlimit`, con database in memoria e autorizzatore SQLite attivo per bloccare chiamate al filesystem o estensioni C.
- **Protezione da *Prompt Injection*:** Il codice e le motivazioni dello studente sono delimitati come meri dati passivi all'interno del contesto inviato all'LLM. Il modello tutor non dispone di tool d'azione, privilegi di esecuzione o accesso alla memoria esterna. Le risposte generate sono analizzate prima dell'invio allo studente per bloccare la fuga di soluzioni.
- **Tracciabilità e logging:** Conservazione dei log di sistema e delle chiamate di inferenza con canali separati per il monitoraggio operativo e la rilevazione tempestiva di eventuali anomalie.
- **Verifica delle licenze:** Accertamento della compatibilità per uso didattico e accademico delle licenze del software open-source e dei pesi dei modelli linguistici adottati (es. licenze Apache 2.0, MIT o specifiche per pesi aperti conformi all'Open Source Initiative).

7.3 Valutazione e quadro regolatorio sull'Intelligenza Artificiale (AI Act)

- **Assenza di decisioni automatizzate e di valutazione algoritmica:** Il sistema è privo di funzioni di calcolo di voti, attribuzione di punteggi sommatori o giudizi di idoneità. Gli esiti dei controlli tecnici deterministici costituiscono pure informazioni diagnostiche formative di processo. La valutazione finale resta un atto esclusivamente umano, esercitato dal docente sulla base di prove individuali in presenza (fase 9) e della lettura qualitativa degli elaborati. Questa scelta garantisce piena conformità con l'art. 22 del GDPR (divieto di decisioni basate unicamente sul trattamento automatizzato).
- **Inquadramento rispetto all'EU AI Act (Regolamento UE 2024/1689):** L'AI Act include tra i sistemi ad alto rischio (Allegato III, punto 3) i sistemi di IA destinati a valutare i risultati dell'apprendimento, anche quando tali esiti orientino il percorso scolastico. Socratic-DB è deliberatamente architettato per sottrarsi a tale perimetro:
  - Non calcola punteggi;
  - Non formula raccomandazioni vincolanti di avanzamento o blocco;
  - Non stila graduatorie o profili predittivi dello studente. La classificazione va in ogni caso formalizzata nella scheda tecnica del sistema con il DPO dell'istituto.
- **Trasparenza e alfabetizzazione (Art. 50 AI Act):** Gli studenti sono esplicitamente informati del fatto che stanno interagendo con un agente artificiale. L'esperienza didattica è integrata da moduli di

alfabetizzazione critica sull'IA, volti a chiarire come funziona il tutor, quali sono i suoi limiti e perché le verifiche tecniche provengono da software deterministico e non dal modello linguistico.
- **Percorso alternativo:** Viene garantita agli studenti e alle famiglie la possibilità di svolgere il medesimo percorso curricolare con modalità tradizionali (esercitazione standard guidata dal docente in aula e su materiale cartaceo/digitale convenzionale) senza alcuna penalizzazione.

7.4 Limiti del sistema e minacce alla validità

- **Allucinazioni e compiacenza dell'LLM (*Sycophancy*):** I modelli linguistici possono produrre risposte errate, incoerenti o accondiscendere a ragionamenti scorretti. Per mitigare tale rischio, l'LLM è confinato al ruolo dialogico e le sue risposte sono vincolate alle diagnosi deterministiche.
- **Rigidità dei controlli deterministici:** I controlli automatici valutano solo ciò che è formalizzato nei test: un controllo eccessivamente stringente rischia di respingere schemi alternativi del tutto validi. Rimane sempre attiva la facoltà di segnalazione di falsi positivi al docente.
- **Limiti espressivi del DDL SQLite:** Vincoli di dominio che coinvolgono tuple multiple o dipendenze temporali complesse non sono esprimibili con semplici vincoli dichiarativi in SQLite e richiedono trigger o controlli applicativi.
- **Dipendenze funzionali incomplete:** La correttezza del calcolo delle forme normali (2NF, 3NF, BCNF) dipende dall'insieme delle dipendenze funzionali dichiarate dallo studente nel modello relazionale; se lo studente omette dipendenze fondamentali, il calcolo algoritmico risulterà parziale.
- **Limiti di scalabilità hardware:** Con una singola GPU locale da 32 GB di VRAM, la capacità di servizio simultaneo è calibrata su una singola classe (5–20 postazioni attive con intervalli di digitazione umana).
- **Automation bias del docente:** Nell'uso dello strumento d'autore (Sezione 5.8), esiste il rischio che il docente approvi la checklist o lo schema proposto dall'IA senza un'attenta rilettura critica. La validazione umana deve essere formalizzata come passaggio obbligatorio e verificata a campione.

8. Protocollo di sperimentazione empirica

Prima di estendere l'adozione del sistema a più istituti o corsi, il progetto prevede una sperimentazione controllata volta a quantificare l'efficacia didattica, l'usabilità e la sostenibilità operativa.

8.1 Fasi e disegno della ricerca

La sperimentazione si articola in sette passaggi metodologici:

1. **Definizione degli obiettivi formativi mirati:** Focalizzazione su un sottoinsieme specifico di competenze modellistiche e interrogative (da O2 a O7: modellazione ER, traduzione in schema relazionale, individuazione di vincoli non banali, calcolo delle forme normali, scrittura di query SQL con aggregazioni e join).
2. **Strumenti di misurazione standardizzati:**
   - *Pre-test diagnostico*: Somministrazione iniziale per rilevare il livello di partenza degli studenti.
   - *Attività formative con il tutor*: Svolgimento delle consegne C1–C4 con tracciamento dettagliato delle revisioni.
   - *Post-test individuale*: Valutazione delle competenze acquisite al termine dell'intervento.
   - *Prova di trasferimento a distanza (3–4 settimane)*: Risoluzione di una traccia inedita senza alcun supporto del tutor (fase 9 / consegna C5), per verificare la ritenzione nel medio termine e l'assenza di dipendenza dall'assistente.
   - *Correzione in cieco*: Valutazione degli elaborati da parte di due docenti indipendenti all'oscuro dell'identità dello studente, con rubrica analitica descrittiva e calcolo dell'indice di accordo inter-valutatore.
3. **Validazione preventiva del tutor (Golden Set):** Prima dell'interazione con gli studenti, il tutor linguistico è sottoposto al golden set di collaudo (Sezione 5.5.3):
   - Almeno 30 casi noti per ciascuna fase;
   - Misurazione della fedeltà ai livelli di aiuto (nessun controesempio a L1, nessun codice risolutivo completo);
   - Calcolo del tasso di mancata fuga della soluzione (obiettivo: 100%);
   - Verifica di non-contraddizione rispetto alle diagnosi deterministiche.
4. **Studio pilota:**
   - Coinvolgimento di una o due classi (5–20 studenti per classe) per un ciclo di 6–8 lezioni curricolari in laboratorio.
   - Disegno di ricerca prevalentemente *intra-soggetto* (confronto pre/post sullo stesso gruppo di studenti).
   - Rilevazione qualitativa continua: osservazione sul campo dei comportamenti degli studenti, registrazione dei blocchi tecnici o didattici, note del docente.
   - Affinamento iterativo dei prompt, dei pesi di quantizzazione e delle regole di validazione prima della sperimentazione su scala più ampia.
5. **Verifica empirica del trasferimento:** Confronto delle prestazioni della prova individuale finale rispetto al pre-test diagnostico iniziale, utilizzando una traccia concettualmente isomorfa ma ambientata in un dominio applicativo differente.
6. **Analisi multidimensionale dei risultati:** Oltre ai punteggi grezzi di correttezza, l'analisi considera:
   - *Grado di autonomia*: Evoluzione della richiesta di livelli di aiuto (tendenza a diminuire il ricorso a L3/L4);
   - *Qualità del ragionamento*: Pertinenza, rigore e ricchezza delle motivazioni scritte nei campi `motivazione`;
   - *Capacità di correzione attiva*: Analisi della cronologia dei commit Git tra il primo tentativo e la consegna validata;

- *Esperienza percepita*: Questionario di fine percorso sull'utilità percepita, senso di autoefficacia e fiducia nel tutor.

7. **Confronto controllato (Fase estesa successiva):** Qualora siano disponibili classi parallele con il medesimo programma curricolare, confronto tra:
   - *Gruppo sperimentale*: Attività svolta con Socratic-DB.
   - *Gruppo di controllo*: Attività tradizionale con correzione manuale periodica del docente. Adozione di schemi a rotazione (*crossover*) per garantire a tutti gli studenti le medesime opportunità formative.

8.2 Raccolta dati e metriche di processo

Tutti i dati raccolti sono associati esclusivamente a identificativi pseudonimi (`s001`, `s002`, ...):

- **Metriche quantitative di sistema**:
  - Numero di tentativi e commit effettuati per ciascuna fase;
  - Frequenza e tipologia delle diagnosi deterministiche sollevate (es. violazioni di integrità referenziale, errori di normalizzazione, query SQL non sintattiche);
  - Livello massimo di aiuto richiesto (L0–L4) per esercizio;
  - Tempo impiegato per completare ciascuna consegna;
  - Tempi effettivi di preparazione e validazione delle tracce da parte del docente (misura della sostenibilità).
- **Metriche di apprendimento**:
  - Punteggi ottenuti nelle rubriche del pre-test, post-test e prova di trasferimento;
  - Indice di qualità delle spiegazioni (rubrica qualitativa a 3 livelli).

8.3 Analisi statistica e criteri di successo

- **Trattamento statistico:**
  - Calcolo di statistiche descrittive (mediana, intervallo interquartile, medie e deviazioni standard);
  - Stima delle dimensioni dell'effetto (*effect size*, es. d di Cohen o r di Wilcoxon) con intervalli di confidenza al 95%;
  - Data la dimensione contenuta dei campioni tipici scolastici (5–20 studenti per classe), l'analisi quantitativa deve essere interpretata con prudenza ed essere corroborata dall'analisi qualitativa dei percorsi individuali.
- **Dichiarazione delle minacce alla validità:**
  - *Effetto novità*: Entusiasmo iniziale per l'uso dell'IA generativa;
  - *Apprendimento esterno*: Studio autonomo a casa o supporto di terzi tra pre-test e post-test;
  - *Maturazione naturale*: Miglioramento fisiologico dovuto al normale avanzamento dell'anno scolastico.

8.3.1 Criteri di successo definiti a priori

1. **Incremento significativo nel post-test e nel trasferimento**: Miglioramento misurabile nella capacità di modellazione e interrogazione autonoma rispetto al livello di ingresso.
2. **Riduzione della dipendenza dall'aiuto**: Diminuzione statisticamente visibile del livello medio di aiuto richiesto tra la prima e l'ultima consegna.
3. **Miglioramento della capacità critica**: Incremento qualitativo dei testi di motivazione inseriti negli schemi.
4. **Sostenibilità del carico di lavoro del docente**: Tempo medio di preparazione e validazione della traccia tramite strumento d'autore contenuto entro i 30 minuti; tempo di revisione per classe compatibile con il normale orario di servizio.
5. **Accordo del tutor sulle query**: Accordo tra il parere non vincolante del tutor in fase 8 e il giudizio dei docenti su almeno l'85% del campione esaminato.
6. **Zero incidenti di sicurezza e privacy**: Nessun dato personale esfiltrato e rispetto integrale dei protocolli approvati.

8.4 Etica, governance e consenso

- **Approvazione formale degli organi collegiali:** Delibera del Collegio dei Docenti e del Consiglio di Istituto sull'adozione della sperimentazione.
- **Coinvolgimento del DPO:** Emissione del parere favorevole sulla DPIA e sulle informative per famiglie e studenti.
- **Volontarietà e diritto di recesso:** Possibilità per ogni studente di partecipare senza obbligo e di ritirarsi in qualsiasi momento optando per il percorso didattico alternativo tradizionale, senza alcuna penalizzazione sul piano valutativo curricolare.
- **Restituzione formativa:** Sessione finale di debriefing con gli studenti per discutere i risultati aggregati, le difficoltà emerse e il ruolo critico dell'IA nei contesti professionali.

Bibliografia

[1] B. A. Becker, P. Denny, J. Finnie-Ansley, A. Luxton-Reilly, J. Prather, e E. A. Santos, «Programming Is Hard - Or at Least It Used to Be: Educational Opportunities and Challenges of AI Code Generation», in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, in SIGCSE 2023. Toronto ON, Canada: Association for Computing Machinery, 2023, pp. 500–506. doi: 10.1145/3545945.3569759.

[2] P. Denny *et al.*, «Computing Education in the Era of Generative AI», *Commun. ACM*, vol. 67, fasc. 2, pp. 56–67, gen. 2024, doi: 10.1145/3624720.

[3] J. Prather *et al.*, «The Robots Are Here: Navigating the Generative AI Revolution in Computing Education», in *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, in ITiCSE-WGR '23. Turku, Finland: Association for Computing Machinery, 2023, pp. 108–159. doi: 10.1145/3623762.3633499.

[4] E. F. Codd, «A relational model of data for large shared data banks», *Commun. ACM*, vol. 13, fasc. 6, pp. 377–387, giu. 1970, doi: 10.1145/362384.362685.

[5] P. P.-S. Chen, «The entity-relationship model–toward a unified view of data», *ACM Trans. Database Syst.*, vol. 1, fasc. 1, pp. 9–36, mar. 1976, doi: 10.1145/320434.320440.

[6] A. Mitrovic, «Experiences in Implementing Constraint-Based Modeling in SQL-Tutor», in *Proceedings of the 4th International Conference on Intelligent Tutoring Systems*, in ITS '98. Berlin, Heidelberg: Springer-Verlag, 1998, pp. 414–423. doi: 10.1007/3-540-68716-5_47.

[7] A. Mitrovic, M. Mayo, P. Suraweera, e B. Martin, «Constraint-Based Tutors: A Success Story», in *Engineering of Intelligent Systems*, L. Monostori, J. Váncza, e M. Ali, A c. di, Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 931–940. doi: 10.1007/3-540-45517-5_103.

[8] D. Miedema, E. Aivaloglou, e G. Fletcher, «Identifying SQL Misconceptions of Novices: Findings from a Think-Aloud Study», in *Proceedings of the 17th ACM Conference on International Computing Education Research*, in ICER 2021. Virtual Event, USA: Association for Computing Machinery, 2021, pp. 355–367. doi: 10.1145/3446871.3469759.

[9] T. Taipalus, M. Siponen, e T. Vartiainen, «Errors and Complications in SQL Query Formulation», *ACM Trans. Comput. Educ.*, vol. 18, fasc. 3, ago. 2018, doi: 10.1145/3231712.

[10] D. E. Miedema, «On Learning SQL: Disentangling concepts in Data Systems Education», Phd Thesis 1 (Research TU/e / Graduation TU/e), Eindhoven University of Technology, 2024.

[11] D. Ponzini, G. Guerrini, e B. Catania, «A Proposal for Revising SQL Error Taxonomies Based on Automated Detection», in *EDBT/ICDT Workshops*, 2026. [Online]. Disponibile su: https://ceur-ws.org/Vol-4192/DataEd-paper9.pdf

[12] D. Miedema *et al.*, «Explanations of Data Systems Concepts in CS Education: An Updated View on Notional Machines», in *Proceedings of the 2025 Working Group Reports on Innovation and Technology in Computer Science Education*, in ITiCSE-WGR 2025. Nijmegen, Netherlands: Association for Computing Machinery, 2026, pp. 108–135. doi: 10.1145/3760545.3783971.

[13] D. Ponzini, B. Catania, e G. Guerrini, «Enhancing SQL Learning Through Generative AI and~Student Error Analysis», in *New Trends in Database and Information Systems*, P. K. Chrysanthis, K. Nørvåg, K. Stefanidis, Z. Zhang, E. Quintarelli, e E. Zumpano, A c. di, Cham: Springer Nature Switzerland, 2026, pp. 118–128. doi: 10.1007/978-3-032-05727-3_12.

[14] D. Ponzini, E. Calza, B. Catania, e G. Guerrini, «Personalized SQL Exercise Generation with~Large Language Models and~Constraint-Based Validation», in *Advances in Databases and Information Systems*, G. Vargas-Solar, K. Stefanidis, T. Palpanas, P. Marcel, e M. Halfeld-Ferrari, A c. di, Cham: Springer Nature Switzerland, 2027, pp. 232–248. doi: 10.1007/978-3-032-39820-8_16.

[15] G. Guerrini e D. Ponzini, «Apprendimento di SQL basato su errori frequenti», in *ITADINFO 2026: Quarto Convegno Italiano sulla Didattica dell'Informatica*, V. Lonati e G. Pozzan, A c. di, Padova: Edizioni Kangourou, 2026, pp. 372–373.

[16] B. S. Bloom, «The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring», *Educational Researcher*, vol. 13, fasc. 6, pp. 4–16, 1984, Consultato: 20 ottobre 2026. [Online]. Disponibile su: http://www.jstor.org/stable/1175554

[17] L. S. VYGOTSKY, *Mind in Society: Development of Higher Psychological Processes*. Harvard University Press, 1978. Consultato: 20 ottobre 2026. [Online]. Disponibile su: http://www.jstor.org/stable/j.ctvjf9vz4

[18] J. Macina *et al.*, «Opportunities and Challenges in Neural Dialog Tutoring», in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, A. Vlachos e I. Augenstein, A c. di, Dubrovnik, Croatia: Association for Computational Linguistics, mag. 2023, pp. 2357–2372. doi: 10.18653/v1/2023.eacl-main.173.

[19] Parlamento europeo e Consiglio dell'Unione europea, «Regolamento (UE) 2016/679 del Parlamento europeo e del Consiglio, del 27 aprile 2016». [Online]. Disponibile su: https://eur-lex.europa.eu/eli/reg/2016/679/oj

[20] Parlamento europeo e Consiglio dell'Unione europea, «Regolamento (UE) 2024/1689 del Parlamento europeo e del Consiglio, del 13 giugno 2024». [Online]. Disponibile su: https://eur-lex.europa.eu/eli/reg/2024/1689/oj